

ASIGNATURA DOSW	TEMAS Patrones de Diseño Combinados	MODALIDAD Individual	NOMBRE / FECHA ____Juan David Munar____
--------------------	---	-------------------------	--

Objetivo: Cada ejercicio presenta un caso real que requiere combinar exactamente 2 patrones de diseño. Para cada uno: identifique el rol de cada patrón, describa cómo interactúan, proponga el esquema de clases resultante y justifique por qué esa combinación es superior a una solución sin patrones.

PATRONES DE DISEÑO DE SOFTWARE - MULTIVARIABLE

Ejercicios de Aplicación

#01 Plataforma de Pagos Inteligentes

Una aplicación de e-commerce permite pagar con tarjeta, PSE, Nequi, PayPal y transferencia bancaria. Cada medio tiene una lógica distinta pero el flujo de compra es el mismo. Además, según el país del usuario, el sistema construye el proveedor de pago correcto (Colombia → PSE/Nequi, USA → PayPal/Stripe).

► Strategy

Strategy encapsula cada algoritmo de pago en una clase independiente (TarjetaStrategy, PseStrategy, NequiStrategy). El Checkout trabaja con la interfaz PaymentStrategy sin importar cuál medio se use.

► Factory Method

Factory Method crea el proveedor correcto según el país del usuario. ColombiaPaymentFactory, UsaPaymentFactory. El cliente no sabe qué objeto concreto se construye.

CÓMO INTERACTÚAN

El usuario selecciona su país → Factory construye el gateway correcto → ese gateway implementa PaymentStrategy → el Checkout llama strategy.process(amount). La Factory decide qué Strategy instanciar; el Checkout nunca cambia.

ESQUEMA DE CLASES SUGERIDO

```
interface PaymentStrategy { void process(double amount); }
class TarjetaStrategy implements PaymentStrategy { ... }
class PseStrategy implements PaymentStrategy { ... }
class NequiStrategy implements PaymentStrategy { ... }

interface PaymentFactory { PaymentStrategy create(String type); }
class ColombiaPaymentFactory implements PaymentFactory { ... }
class UsaPaymentFactory implements PaymentFactory { ... }
```

 **Pista:** Strategy resuelve 'cómo pagar'; Factory resuelve 'quién construye el que paga'. Son responsabilidades distintas. Sin Factory, el Checkout conocería qué Strategy instanciar — acoplamiento.

#02 Sistema de Notificaciones Multicanal

Cuando un pedido cambia de estado (pendiente → enviado → entregado), el sistema notifica por correo, SMS, WhatsApp y push. No todos los usuarios tienen activos los mismos canales. Cada canal tiene su propia forma de construir y formatear el mensaje.

► Observer

Observer desacopla el pedido de los canales. El Pedido es el Subject. EmailNotifier, SmsNotifier y PushNotifier son Observers. Agregar un canal nuevo no modifica el Pedido.

► Factory Method

Factory Method crea el mensaje correcto para cada canal. EmailMessageFactory genera HTML. SmsMessageFactory genera texto plano de 160 chars. PushMessageFactory genera payload JSON.

🔗 CÓMO INTERACTÚAN

El Pedido cambia de estado → notifica a todos los Observers activos → cada Observer llama a su Factory para construir el mensaje correcto para ese canal → envía.

ESQUEMA DE CLASES SUGERIDO

```
interface NotificationObserver { void notify(OrderEvent event); }
class EmailNotifier implements NotificationObserver { ... }
class SmsNotifier implements NotificationObserver { ... }
class PushNotifier implements NotificationObserver { ... }

interface MessageFactory { Message build(OrderEvent event); }
class EmailMessageFactory implements MessageFactory { ... }
class SmsMessageFactory implements MessageFactory { ... }
```

💡 **Pista:** Observer = desacoplamiento del emisor. Factory = desacoplamiento de la construcción del mensaje. Sin Factory, cada Observer construiría el mensaje a su manera con lógica dispersa.

#03 Sistema de Reportes Empresariales

La empresa genera reportes en PDF, Excel y CSV. Todos siguen los mismos 4 pasos: obtener datos → procesar información → aplicar formato → exportar archivo. Pero cada formato implementa 'aplicar formato' y 'exportar' de forma diferente. Además, el sistema decide dinámicamente qué tipo de reporte crear.

► Template Method

Template Method define la estructura del algoritmo en la clase base ReportGenerator con un método final generate() que llama en orden los 4 pasos. Las subclases

► Factory Method

Factory Method crea la instancia correcta según la solicitud. ReportFactory.create('PDF') retorna PdfReport.

sobreescriben solo los pasos variables (applyFormat, export).

ReportFactory.create('CSV') retorna CsvReport. El cliente no instancia directamente.

🔗 CÓMO INTERACTÚAN

El cliente pide 'reporte PDF' → Factory construye PdfReport → el cliente llama report.generate() → Template Method ejecuta los 4 pasos en orden, usando la implementación PDF para los pasos variables.

ESQUEMA DE CLASES SUGERIDO

```
abstract class ReportGenerator {
    public final void generate() { // Template Method
        fetchData();           // paso fijo
        processData();          // paso fijo
        applyFormat();          // abstracto - cada formato lo implementa
        exportFile();           // abstracto - cada formato lo implementa
    }
    abstract void applyFormat();
    abstract void exportFile();
}
class PdfReport extends ReportGenerator { ... }
class ExcelReport extends ReportGenerator { ... }
class CsvReport extends ReportGenerator { ... }
```

💡 **Pista:** Template Method = 'el esqueleto es fijo, los detalles varían'. Strategy = 'el algoritmo completo es intercambiable'. Si el proceso tiene pasos fijos y variables, Template Method; si todo el algoritmo es intercambiable, Strategy.

#04 Plataforma de Videojuegos — Personajes

Un videojuego crea guerreros, magos y arqueros. Cada personaje puede tener habilidades especiales, armadura, arma y mejoras temporales (escudo de hielo, velocidad extra, invisibilidad). El personaje se construye al inicio de la partida, pero sus poderes pueden aumentar dinámicamente durante el juego.

► Builder

Builder construye el personaje paso a paso al inicio. WarriorBuilder permite setArmor().setWeapon().setSkill(). El Director puede construir arquetipos predefinidos (guerrero elite, mago de fuego). Evita constructores con 10 parámetros.

► Decorator

Decorator agrega poderes dinámicamente sin modificar la clase base del personaje. ShieldDecorator, SpeedDecorator e InvisibilityDecorator envuelven el personaje y añaden comportamiento en runtime.

🔗 CÓMO INTERACTÚAN

Builder crea el personaje base configurable → durante la partida, Decorator envuelve el personaje con poderes temporales → al terminar el efecto, el wrapper se descarta sin afectar la clase base del personaje.

ESQUEMA DE CLASES SUGERIDO

```
// Builder: construcción al inicio
WarriorBuilder builder = new WarriorBuilder();
Character warrior = builder.setArmor("steel")
    .setWeapon("sword")
    .setSkill("rage")
    .build();

// Decorator: poderes durante la partida
Character powered = new ShieldDecorator(
    new SpeedDecorator(warrior));
powered.attack(); // ejecuta: velocidad extra + escudo + ataque base
```

Pista: Sin Decorator: $2^5 = 32$ subclases para 5 poderes combinables. Con Decorator: 5 wrappers + 1 base = 6 clases. La explosión combinatoria es la señal inequívoca para usar Decorator.

#05 Integración con Sistema Bancario Antiguo

El sistema moderno usa PaymentProcessor con métodos modernos. El banco antiguo expone LegacyBankService con métodos incompatibles (executeTransaction, verifyBalance en centavos). Además, usar LegacyBankService directamente requiere 8 pasos de inicialización que los desarrolladores no deberían conocer.

► Adapter

Adapter hace que LegacyBankService sea compatible con la interfaz PaymentProcessor. LegacyBankAdapter implementa PaymentProcessor e internamente traduce las llamadas: amount → cents, pay() → executeTransaction().

► Facade

Facade expone un método simple procesarPago(monto) que internamente orquesta los 8 pasos de inicialización y uso del LegacyBankService (o del Adapter). Los desarrolladores usan la Facade y no conocen los detalles.

🔗 CÓMO INTERACTÚAN

Desarrollador llama BankFacade.procesarPago(monto) → Facade inicializa conexión, sesión y contexto → delega al LegacyBankAdapter → Adapter traduce al formato legacy → LegacyBankService ejecuta. El desarrollador nunca toca LegacyBankService.

ESQUEMA DE CLASES SUGERIDO

```
// Adapter: traduce la interfaz
class LegacyBankAdapter implements PaymentProcessor {
    private LegacyBankService legacy;
    public void pay(double amount) {
        int cents = (int)(amount * 100); // traducción
        legacy.executeTransaction("ACC", cents);
    }
}

// Facade: simplifica el uso
```

```
class BankFacade {
    public void procesarPago(double monto) {
        // 8 pasos de inicialización...
        adapter.pay(monto);
    }
}
```

Pista: Adapter = 'hablar el idioma del otro'. Facade = 'no me cuentes todo, dame lo simple'. Puedes tener ambos: la Facade usa el Adapter internamente. Son complementarios, no excluyentes.

#06 Motor de Recomendaciones

Una plataforma tipo Netflix usa algoritmos de recomendación por género, historial, popularidad y similitud con otros usuarios. El usuario puede cambiar sus preferencias de recomendación en cualquier momento. Cuando esto ocurre, la página principal, las notificaciones y la lista de 'sugeridos' deben actualizarse automáticamente.

► Strategy

Strategy permite intercambiar el algoritmo de recomendación en tiempo de ejecución. GenreStrategy, HistoryStrategy, PopularityStrategy y SimilarityStrategy implementan RecommendationAlgorithm. El motor cambia de algoritmo sin reiniciar.

► Observer

Observer notifica automáticamente a todos los componentes cuando cambian las preferencias. HomePageComponent, NotificationService y SuggestedListComponent son Observers del evento 'preferencias cambiadas'.

🔗 CÓMO INTERACTÚAN

Usuario cambia preferencias → el perfil de usuario es el Subject, notifica a sus Observers → cada Observer reactualiza su contenido usando el nuevo algoritmo Strategy que fue configurado → la UI se actualiza sin polling.

ESQUEMA DE CLASES SUGERIDO

```
interface RecommendationAlgorithm {
    List<Content> recommend(User user);
}

class GenreStrategy implements RecommendationAlgorithm { ... }
class HistoryStrategy implements RecommendationAlgorithm { ... }
class PopularityStrategy implements RecommendationAlgorithm { ... }

// Observer: reacciona al cambio
interface PreferenceObserver { void onPreferenceChanged(User user); }
class HomePageComponent implements PreferenceObserver { ... }
class SuggestedListComponent implements PreferenceObserver { ... }
```

Pista: Strategy responde 'cómo recomendar'; Observer responde 'a quién avisar que cambió el cómo'. Son ortogonales: cambiar el algoritmo (Strategy) dispara el aviso (Observer) a los componentes que deben re-renderizarse.

#07 Flujo de Aprobación de Documentos

Los documentos pasan por: revisión del autor, revisión del líder, revisión jurídica, revisión financiera y aprobación final. No todos pasan por todas las etapas. Además, el documento tiene estados propios: borrador, en revisión, aprobado, rechazado. La transición de estado depende del resultado de cada handler de la cadena.

► Chain of Responsibility

Chain of Responsibility encadena los validadores. Cada handler (AutorHandler, LiderHandler, JuridicoHandler) decide si procesa el documento o lo pasa al siguiente. La cadena puede configurarse distinto según el tipo de documento.

► State

State maneja las transiciones de estado del documento. DraftState, InReviewState, ApprovedState, RejectedState. Cada estado sabe a qué estado puede transicionar y qué operaciones permite. Elimina los switch/if de estado.

🔗 CÓMO INTERACTÚAN

Un handler de la cadena procesa el documento → según su resultado, invoca document.approve() o document.reject() → el objeto State actual del documento ejecuta la transición correspondiente → el documento nunca tiene un switch de estados.

ESQUEMA DE CLASES SUGERIDO

```
// Chain: cada handler decide si pasa al siguiente
abstract class DocumentHandler {
    private DocumentHandler next;
    public void handle(Document doc) {
        if (canHandle(doc)) process(doc);
        else if (next != null) next.handle(doc);
    }
    abstract boolean canHandle(Document doc);
    abstract void process(Document doc);
}

// State: el documento delega comportamiento a su estado
interface DocumentState { void approve(Document doc); void reject(Document doc); }
class InReviewState implements DocumentState { ... }
class ApprovedState implements DocumentState { ... }
```

💡 **Pista:** Sin State: cada método de Document tendría switch(estado){case DRAFT: ... case IN_REVIEW:...}. Con State: cada estado encapsula su propio comportamiento. El documento no sabe en qué estado está — su estado sabe qué hacer.

#08 Sistema de Pedidos en Restaurante

El cliente construye una hamburguesa eligiendo ingredientes, tamaño, tipo de pan, acompañamientos y extras. Después de confirmado el pedido, el sistema debe notificar a cocina (preparar), a facturación (generar cuenta) y al domiciliario (preparar ruta) sin que el pedido los conozca directamente.

► Builder

Builder construye el pedido personalizado paso a paso. OrderBuilder permite setSize().setMeat().setToppings().addSide(). El pedido resultante es inmutable una vez construido. Evita el constructor caótico con todos los ingredientes.

► Observer

Observer notifica a los subsistemas cuando el pedido se confirma. KitchenService, BillingService y DeliveryService son Observers. El pedido solo hace pedido.confirm() — no sabe a quién avisar.

🔗 CÓMO INTERACTÚAN

Cliente configura el pedido con Builder → llama build() que retorna un Order inmutable → el sistema hace order.confirm() → el Order notifica a todos sus Observers → cada subsistema reacciona de manera independiente.

ESQUEMA DE CLASES SUGERIDO

```
Order order = new OrderBuilder()
    .setSize(Size.LARGE)
    .setMeat(Meat.DOUBLE_BEEF)
    .addTopping("queso", "lechuga")
    .addSide("papas", "gaseosa")
    .build(); // Order inmutable

order.addObserver(new KitchenService());
order.addObserver(new BillingService());
order.addObserver(new DeliveryService());
order.confirm(); // notifica a los 3 observers
```

💡 **Pista:** Builder garantiza que el pedido esté completo y válido antes de existir (invariantes en build()). Observer garantiza que la confirmación desencadene reacciones sin acoplamiento. Son momentos distintos del ciclo de vida del pedido.

#09 Sistema de Autenticación Empresarial

La empresa tiene 5 métodos de autenticación: usuario/contraseña, Google, Microsoft, token empresarial y biometría. Según el tipo de usuario, el sistema selecciona el mecanismo correcto. Una vez autenticado, la solicitud pasa por: validación de credenciales, validación de permisos, validación de ubicación y validación de horario laboral.

► Strategy

Strategy selecciona el mecanismo de autenticación. PasswordStrategy, GoogleStrategy, BiometricStrategy implementan AuthStrategy. El AuthService recibe la

► Chain of Responsibility

Chain of Responsibility procesa las validaciones en secuencia. CredentialValidator → PermissionValidator → LocationValidator

estrategia correcta según el tipo de usuario y llama `authenticate()`.

→ `TimeValidator`. Cada uno decide si pasa al siguiente o lanza un `AccessDeniedException`.

🔗 CÓMO INTERACTÚAN

Usuario intenta acceder → `AuthService` selecciona la `Strategy` correcta → autenticación exitosa → el resultado pasa por la cadena de validadores → si todos aprueban, se concede acceso. `Strategy` decide 'cómo autentico'; `Chain` decide 'si tengo acceso'.

ESQUEMA DE CLASES SUGERIDO

```
// Strategy: mecanismo de autenticación
interface AuthStrategy { AuthResult authenticate(Credentials c); }
class PasswordStrategy implements AuthStrategy { ... }
class GoogleStrategy implements AuthStrategy { ... }
class BiometricStrategy implements AuthStrategy { ... }

// Chain: validaciones post-autenticación
CredentialValidator cred = new CredentialValidator();
PermissionValidator perm = new PermissionValidator();
LocationValidator loc = new LocationValidator();
cred.setNext(perm).setNext(loc); // cadena encadenada
```

💡 **Pista:** `Strategy` = 'qué llave uso para entrar'. `Chain` = 'pasar los controles de seguridad después de entrar'. Son fases distintas: autenticación (quién eres) → autorización (qué puedes hacer).

#10 Aplicación de Edición de Imágenes

La app permite aplicar filtros acumulativos: blanco y negro, sepia, brillo, contraste y reducción de ruido. El usuario puede aplicar varios filtros sobre la misma imagen en cualquier orden. Además, cada acción debe poder deshacerse de manera individual (no solo deshacer la última).

► Decorator

`Decorator` aplica filtros de forma acumulativa. `GrayscaleDecorator`, `SepiaDecorator`, `BrightnessDecorator` envuelven la imagen. Se pueden apilar en cualquier orden. Agregar un filtro nuevo no modifica los existentes.

► Command

`Command` encapsula cada operación del usuario como un objeto (`ApplyFilterCommand`, `RemoveFilterCommand`). El historial de comandos permite `undo` individual. El comando tiene `execute()` y `undo()`.

🔗 CÓMO INTERACTÚAN

Usuario aplica filtro → se crea un `ApplyFilterCommand` que envuelve la imagen con un `Decorator` → el comando se agrega al historial → usuario hace `undo` → el `Command` quita el `Decorator` de la cadena y restaura la imagen anterior.

ESQUEMA DE CLASES SUGERIDO


```
// Decorator: filtros acumulativos
interface Image { BufferedImage render(); }
class BaseImage implements Image { ... }
class GrayscaleDecorator extends ImageDecorator { ... }
class SepiaDecorator extends ImageDecorator { ... }

// Command: operación reversible
interface ImageCommand { void execute(); void undo(); }
class ApplyFilterCommand implements ImageCommand {
    public void execute() { image = new SepiaDecorator(image); }
    public void undo() { image = image.getWrapped(); }
}
List<ImageCommand> history = new ArrayList<>();
```

 **Pista:** Command para undo: necesitas dos stacks — uno de 'ejecutados' y uno de 'deshechos'. Decorator para filtros: la imagen base nunca cambia, solo se envuelve. Undo = quitar el último wrapper. Son el complemento perfecto.



CRITERIOS DE ENTREGA

Por cada ejercicio

#	CRITERIO	PESO	✓
1	Explicación del rol de cada patrón (para qué sirve específicamente en ese caso).	5%	☐
2	Descripción de la interacción entre los dos patrones (cómo se complementan).	5%	☐
3	Esquema de código que ilustre la solución propuesta.	40%	☐
4	Demostración de ejecución de código - funcional	50%	